Experiment 6 Siddhant Bajaj

Aim:To create, compile, and deploy a smart contract using Solidity and Remix IDE, and to execute transactions for storing, updating, and retrieving data on a simulated Ethereum blockchain.

Objectives

1. To understand the fundamentals of smart contracts and blockchain technology.
2. To write a smart contract using Solidity.
3. To compile the smart contract using Remix IDE.
4. To deploy the contract in a simulated blockchain environment.
5. To execute transactions and modify the contract's state.
6. To retrieve and verify stored data using smart contract functions.

Software Requirements

- **Operating System:** Windows, Linux, or macOS
- **Web Browser:** Google Chrome or any compatible browser
- **Development Environment:** Remix IDE
- **Programming Language:** Solidity
- **Execution Environment:** Remix JavaScript VM

Theory

Blockchain

Blockchain is a distributed digital ledger that records transactions in a sequence of blocks. Each block is linked to the previous block using cryptographic techniques. Blockchain technology provides transparency, data integrity, and resistance to unauthorized modification.

Smart Contract

A smart contract is a computer program deployed on a blockchain that executes according to predefined rules. It can store data, process transactions, and perform operations without requiring a centralized intermediary.

Smart contracts are commonly used in decentralized applications, digital asset management, supply chain tracking, and automated financial operations.

Solidity

Solidity is a high-level, statically typed programming language used to develop smart contracts for the Ethereum Virtual Machine (EVM). It supports variables, functions, data types, conditional statements, and other programming constructs required to implement blockchain applications.

Remix IDE

Remix IDE is a web-based development environment used to write, compile, deploy, and test Solidity smart contracts. It provides a code editor, compiler, deployment interface, and transaction logs.

It also provides the Remix JavaScript VM, which allows developers to test smart contracts in a simulated blockchain environment without using real cryptocurrency.

Transactions in Smart Contracts

A transaction is an operation submitted to a blockchain that can change its state. For example, calling a function that updates a state variable requires a state-changing transaction.

In this experiment, the `setNumber()` function updates the stored numerical value. The `getNumber()` function retrieves the stored value without modifying it.

State Variables and Functions

A state variable stores information in a smart contract's persistent storage. In Solidity, the `uint256` data type represents an unsigned 256-bit integer.

The `public` keyword allows external access and automatically generates a getter function for a public state variable. The `view` keyword indicates that a function reads contract state without modifying it.

In this experiment, the contract stores a number and allows it to be updated and retrieved through functions.

Program

File name: `MyContract.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MyContract {
    uint256 public number;

    // Function to store or update a number
    function setNumber(uint256 _number) public {
        number = _number;
    }
```

```
    // Function to retrieve the stored number
    function getNumber() public view returns (uint256) {
        return number;
    }
}
```

Procedure

1. Open Remix IDE by visiting https://remix.ethereum.org/.
2. Create a new file named `MyContract.sol` in the contracts folder.
3. Write the Solidity program in the editor.
4. Open the Solidity Compiler panel.
5. Select a compatible Solidity compiler version and click **Compile MyContract.sol**.
6. Open the Deploy & Run Transactions panel.
7. Select the Remix JavaScript VM as the execution environment.
8. Select `MyContract` from the contract dropdown.
9. Click **Deploy** to deploy the contract.
10. Expand the deployed contract under the Deployed Contracts section.
11. Click `number` or `getNumber` to retrieve the initial value.
12. Enter `100` in the input field for `setNumber()` and execute the function.
13. Call `getNumber()` to verify that the stored value is `100`.
14. Execute `setNumber()` again with the value `250`.
15. Call `getNumber()` to verify that the stored value is `250`.
16. Observe the transaction details in the Remix terminal and record the results.

Expected Output

After deploying the contract, the initial value of `number` is `0`.

Operation 1: Retrieve the initial value

Input: None
Output: 0

Operation 2: Store the value 100

Function: setNumber(100)
Transaction: Successful

After calling `getNumber()`:

Output: 100

Operation 3: Update the value to 250

Function: setNumber(250)
Transaction: Successful

After calling `getNumber()`:

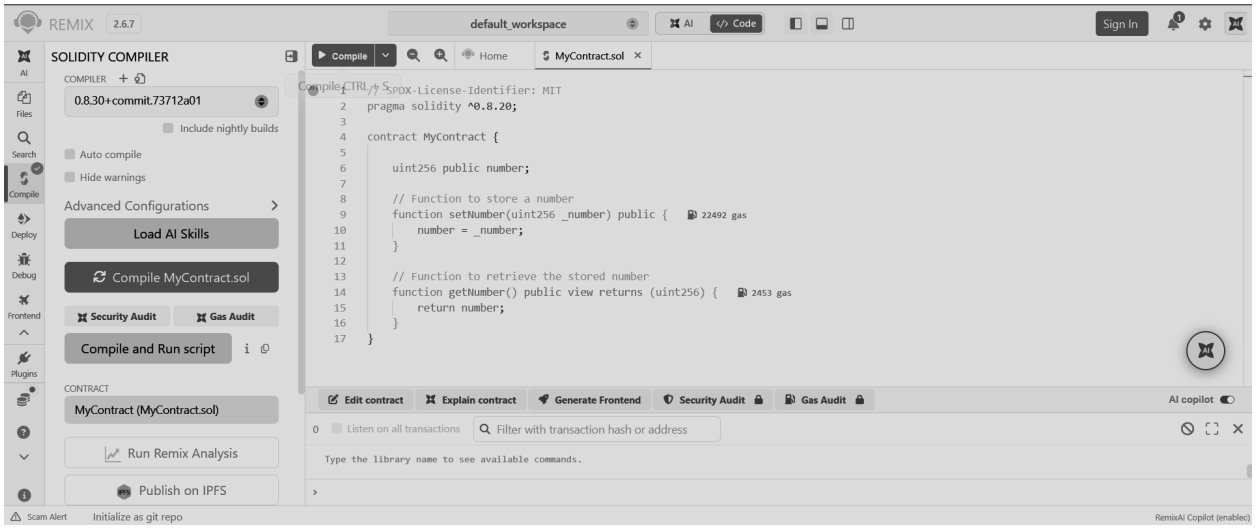
Output: 250

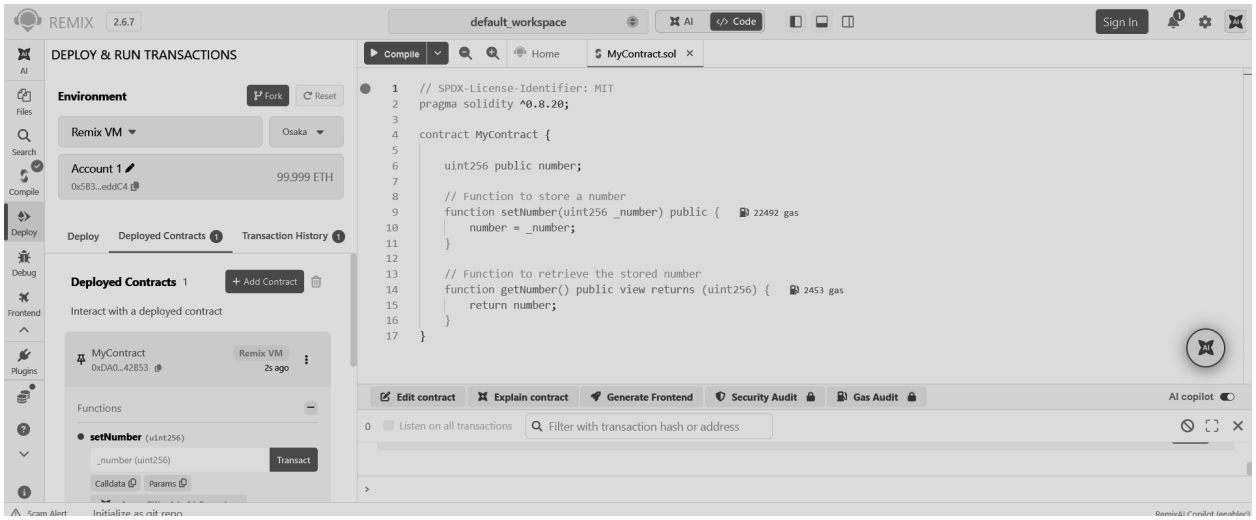
Result

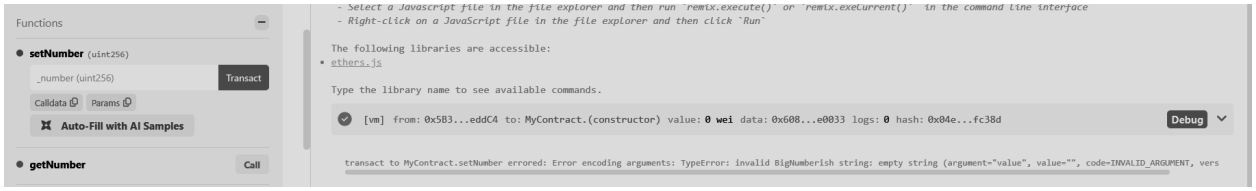
The smart contract was created using Solidity, compiled successfully, and deployed using Remix IDE. Transactions were executed to store and update numerical values, and the results were verified using the getter functions.

Conclusion

Thus, the creation, compilation, and deployment of a smart contract using Solidity and Remix IDE were performed successfully. Transactions were executed to modify the contract's state, and the stored values were retrieved and verified. The experiment demonstrated the fundamental concepts of smart contract development, deployment, transaction execution, and state management in a simulated Ethereum blockchain environment.









[vm] from: 0x5B3...eddC4 to: MyContract.setNumber(uint256) 0xDA0...42B53 value: 0 wei data: 0x3fb...00064 logs: 0 hash: 0xc47...e4eac

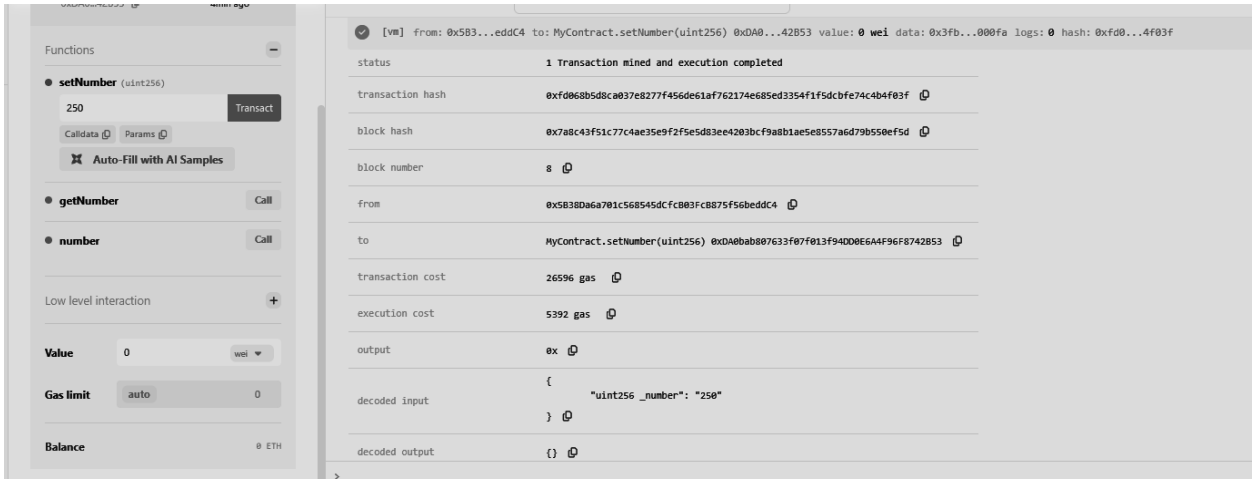
status	1 Transaction mined and execution completed
transaction hash	0xc471282c30287b276aa726c784443184de013ee0bdbb76ff781f6eeab22e4eac
block hash	0xddbb30b96b54b95f962fc276967579e90a75362f20a4157152823edfdeade993
block number	7
from	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to	MyContract.setNumber(uint256) 0xDA0bab807633f07f013f94DD0E6A4F96F8742B53
transaction cost	43696 gas
execution cost	22492 gas
output	0x
decoded input	{ "uint256 _number": "100" }
decoded output	{}

CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: MyContract.number() data: 0x838...1f58a

from	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to	MyContract.number() 0xDA0bab807633f07f013f94DD0E6A4F96F8742B53
execution cost	2424 gas (Cost only applies when called by a contract)
input	0x838...1f58a
output	0x0000000000000000000000000000000000000000000000000000000000000064
decoded input	{}
decoded output	{ "0": "uint256: 100" }
logs	[]
raw logs	[]

CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: MyContract.getNumber() data: 0xf2c...9ecd8

from	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to	MyContract.getNumber() 0xDA0bab807633f07f013f94DD0E6A4F96F8742B53
execution cost	2453 gas (Cost only applies when called by a contract)
input	0xf2c...9ecd8
output	0x0000000000000000000000000000000000000000000000000000000000000064
decoded input	{}
decoded output	{ "0": "uint256: 100" }
logs	[]
raw logs	[]



CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: MyContract.number() data: 0x838...1f58a

from	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to	MyContract.number() 0xDA0bab807633f07f013f94DD0E6A4F96F8742B53
execution cost	2424 gas (Cost only applies when called by a contract)
input	0x838...1f58a
output	0x00000000000000000000000000000000000000000000000000000000000000fa
decoded input	{}
decoded output	{ "0": "uint256: 250" }
logs	[]
raw logs	[]

CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: MyContract.getNumber() data: 0xf2c...9ecd8

from	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to	MyContract.getNumber() 0xDA0bab807633f07f013f94DD0E6A4F96F8742B53
execution cost	2453 gas (Cost only applies when called by a contract)
input	0xf2c...9ecd8
output	0x00000000000000000000000000000000000000000000000000000000000000fa
decoded input	{}
decoded output	{ "0": "uint256: 250" }
logs	[]
raw logs	[]